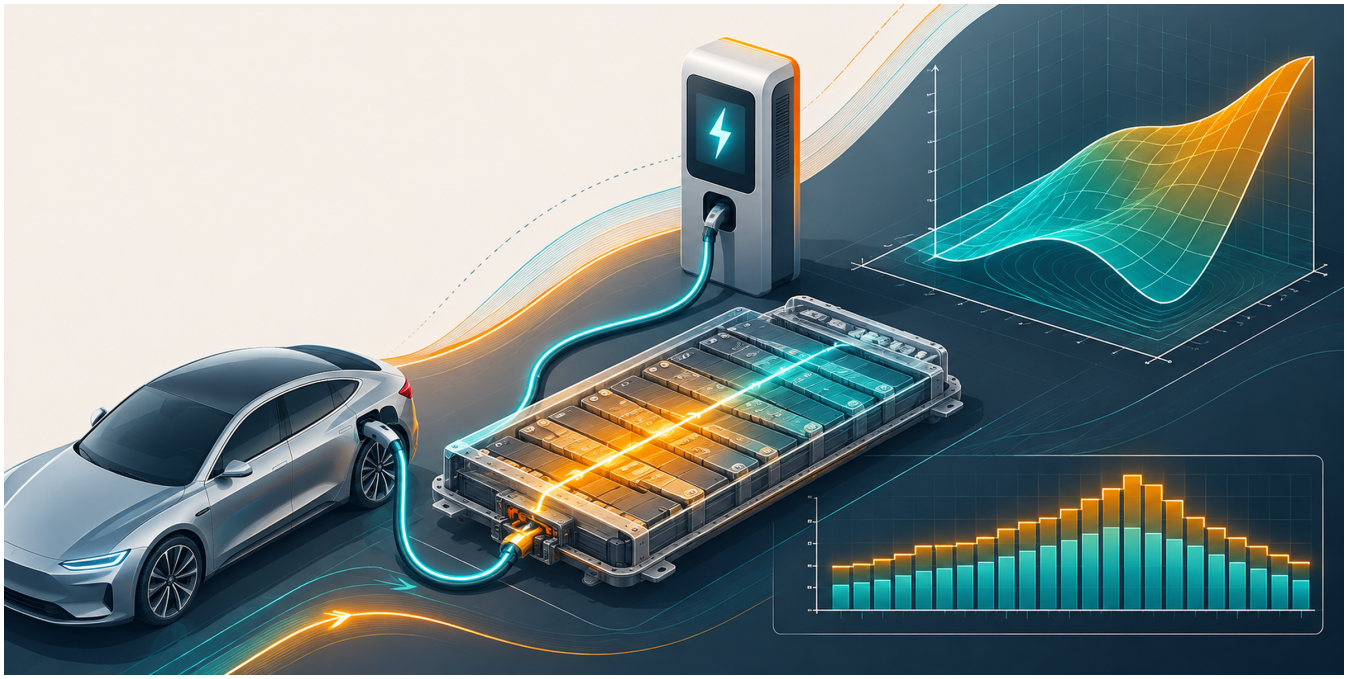
PACEVAL / WP_039 / SCIENTIFIC WHITEPAPER

Battery Charging Cost and Aging Terms with paceval

Reusable mathematical charging and degradation-surrogate expressions inside an audited C++ scheduling workflow



Research outcome. 23 original paceval graphs, 2,231,759 actual Multi-Graph-Batch values and 36 feasible synthetic charging schedules.

Scientific boundary. Charging prices, driving traces, temperatures and the aging-cost coefficients are synthetic. This work validates mathematical feasibility and repeatability; it does **not** quantify physical capacity fade, real charging savings or industrial deployment.

All scientific plots are generated from the actual C++/paceval results. The cover is conceptual artwork, not an observed battery or certified equipment design.

1. Industrial application and current research

Electric-vehicle charging orchestration spans two different time scales: electricity expense is paid per charging session, while electrochemical aging accumulates over many cycles and calendar days. A practical charging service must satisfy a departure requirement, availability windows and battery constraints while comparing alternative mathematical charging policies.

The supplied WP_039 proposal isolates a quadratic, time-indexed cost callback as the reproducible starting point. It deliberately does not claim that a simple quadratic term is a physical battery aging model. This project expands the mathematical kernel while preserving the host optimization boundary: a C++ dynamic-programming planner consumes the **actual paceval-evaluated** cost functions and state transitions.

Research motivation grounded in external literature

Nnorom et al. (2026) examine the tension between hourly operating objectives and multi-year aging via optimization. Appleton and Fotouhi (2023) study charging/degradation tradeoffs under electrical and thermal constraints. NLR/NREL BLAST includes empirically informed temperature, SOC and cycling effects, far beyond the present toy penalty. EVI-Pro characterizes real travel-demand requirements. These references motivate the structure but their empirical models and data are **not** silently imported into this study [1-5].

Question	What is actually demonstrated?
Mathematical execution	Persistent reusable expressions for energy, costs, constrained SOC dynamics, surrogate stress and gradients.
Optimization	Cost-minimal paths on a small 36-state, 24-hour lattice using paceval-computed edge values.
Physical aging	NOT modeled or predicted. Synthetic penalty coefficients are not capacity-fade or SOH calibration.
Industrial reality	No OEM charger controls, measured cell temperature, live tariffs or field validation.

The central technical result is a tested, inspectable separation between a programmable mathematical engine and deterministic charging orchestration, not a superiority claim over native numerical or battery-management systems.

2. Defined industrial use case and system boundary

Synthetic but physically consistent 24-hour scheduling contract

Quantity	Frozen input / rule
Nominal battery energy	64.4 kWh
AC energy to battery	Constant assumed efficiency of 0.92
Charger actions	0, 2, 4, 6, 8 or 10 kW; one-hour intervals
Availability	Hours 0-7 and 18-23; other hours cannot charge
Driving events	9.20 kWh at hour 8 and 5.52 kWh at hour 18
SOC limits and target	15%-92% operating interval; 80% at hour 24
Departure policy	SOC before 08:00 must be at least 20/35
Input scenarios	6 deterministic synthetic time-of-use tariffs and temperatures
Cost policies	Aging weights 0, 8, 30 EUR per synthetic risk unit, and two formula versions

Processing stages

1. The host generates repeatable tariff/travel/cell-temperature inputs. **2.** Paceval graph handles are created once per run. **3.** Multi-Graph batches evaluate proposed 24-hour state/power transitions and 3D chart surfaces. **4.** C++ rejects infeasible transitions and carries out dynamic programming. **5.** Independent C++/mpmath checks verify numerical calculations and energy-balance invariants. **6.** Only executed data are exported as scientific charts.

All claims are relative to the declared **discrete** model. Optimizing this contract is not a guarantee of a globally optimal continuous-time policy or a safe vehicle control command.

3. Mathematical model: energy and state of charge

Let P be charger input power in kW, Δt the slot length in hours, C the energy capacity in kWh, D the fixed driving-consumption event in kWh and s the dimensionless state of charge. All charging and driving energy balances are evaluated with persistent paceval graphs.

Purchased electrical energy and stored electrical energy

$$E_{\mathrm{grid},\,t} = P_t \Delta t, \qquad E_{\mathrm{stored},\,t} = \eta P_t \Delta t$$

Temporal state transition including prescribed driving

$$s_{t+1} = s_t - \frac{D_t}{C} + \frac{\eta P_t \Delta t}{C}$$

The application has a 35-interval SOC lattice (36 states). Because a 2 kW charging action stores 1.84 kWh per hour and 1/35 of 64.4 kWh is exactly 1.84 kWh, every permitted state transition remains on this discrete lattice. This exact alignment is intentional and is verified on each selected path.

Tariff charge and physically meaningful feasibility gates

$$C_{\mathrm{elec},\,t} = c_t P_t \Delta t$$

$$s_{\mathrm{postdrive},\,t} \geq s_{\min}, \quad s_{t+1} \leq s_{\max}, \quad 0 \leq P_t \leq P_{\max}$$

The engine directly computes the margins for post-drive SOC, next-step upper SOC and remaining charger power. The host enforces nonnegative margins and availability as discrete decisions, with a further 08:00 departure reserve and 24-hour terminal SOC equality.

4. Mathematical model: explicitly synthetic aging terms

The original quadratic regularizer is preserved, but it is separated from two independent, non-calibrated stress terms. This distinction prevents a common misinterpretation: a power penalty is a mathematical preference, not an empirically established loss of usable battery capacity.

Cycle-like and calendar-like risk per slot

$$R_{\mathrm{cy},t} = k_{\mathrm{cy}}\left(\frac{P_t}{10\,\mathrm{kW}}\right)^2 \Delta t$$

$$R_{\mathrm{cal},t} = k_{\mathrm{cal}} s_{\mathrm{mid},t}^2 \exp\left[\alpha(T_t - 25)\right]\Delta t$$

The constants k_{cy}=0.020, k_{cal}=0.003 and α=0.040/°C are user-defined synthetic parameters per hour, not inferred from a cell or fleet dataset. The model distinguishes cycle- and calendar-style functions but does not estimate SOH, loss of lithium inventory, calendar lifetime, battery replacement economics or charge acceptance.

Two independently preserved objective versions

$$J_t^{(1)} = c_t P_t \Delta t + w_a(R_{\mathrm{cy},t} + R_{\mathrm{cal},t}) + w_s(P_t/10)^2 \Delta t$$

$$J_t^{(2)} = c_t P_t \Delta t + w_a(R_{\mathrm{cy},t} + R_{\mathrm{cal},t})(1 + 0.2 s_{\mathrm{mid},t}) + w_s(P_t/10)^2 \Delta t$$

Both expressions are real paceval computation handles, with 0/8/30 EUR per synthetic risk unit used as explicit what-if valuations. The second version introduces an SOC-weighted policy multiplier; the original function remains independently callable.

5. Execution: maximizing the actual paceval mathematical role

The C++ application links the original, unchanged **paceval_main.h** and **paceval_main.cpp** interface to the original Linux runtime. Twenty-three mathematical functions are created once; repeated evaluations use **paceval_dGetMultipleComputationsResultsExt** on the same handles.

Graph family	Real execution owner
Energy / transitions (6)	paceval: grid/stored/driving energy, post-drive/midpoint/next SOC
Tariff and aging (6)	paceval: tariff charge, synthetic cycle/calendar/total risk, age cost, quadratic smoothing
Two complete costs (2)	paceval: version 1 and independently retained version 2
Constraint margins (3)	paceval: lower post-drive SOC, upper next SOC, charger margin
Carbon proxy and derivatives (6)	paceval: synthetic grid-carbon term and marginal tariff, cycle, calendar, smooth, full objective
Scheduler and reductions	C++: deterministic traces, feasible-state filtering, path selection, reporting and CSV
Independent numerical checks	C++ native reference plus mpmath 80-digit arithmetic

Batch execution and error semantics

Each input vector contains 17 declared model variables and every persistent handle shares the same variable declaration. The numerical result buffer is interpreted in graph-major order. The SDK batch return is treated as an error flag for this tested Linux library and **every individual returned error code and nonfinite output is checked**. Creation errors are checked separately.

The machine-readable **results/math_execution_inventory.csv** records the exact execution owner and API for each graph. Dynamic programming, random-number generation, sampling, selection, data movement and plotting are not mislabeled as paceval functionality.

6. Defined optimizer and complete run conditions

A discrete constrained global search over the declared lattice

The host searches 24 one-hour stages and 36 SOC states with six charging actions per stage. Every candidate mathematical edge is evaluated by paceval, including state progression, both objectives, all SOC margins, energy and marginal cost functions. C++ dynamic programming then chooses the minimum-cost feasible edge sequence for the chosen synthetic tariff and policy version.

At the end of every run the program reconstructs the selected trajectory and verifies the exact terminal state, time-dependent charger availability, the departure reserve, nonnegative SOC margins and the kWh balance. This is an optimum on the finite declared state/action grid only.

Declared research-workload dimensions

Dimension	Count / value
Tariff cases	6 fixed synthetic profiles
Risk valuations	3 explicit synthetic aging weights
Formula versions	2 persistent independent mathematical cost functions
Completed schedule solutions	36 valid trajectories
Hourly solution output rows	864 (36 schedules × 24 steps)
2D mathematical surface	61 × 61 = 3,721 graph-evaluated points
Model scale	23 persistent paceval formula handles × every tested input

Energy balance as an independent scientific invariant

$$64.4 \cdot \frac{17}{35} + 0.92 \cdot 38.0 - 14.72 = 64.4 \cdot 0.80$$

All 36 schedules satisfy this 38.00 kWh grid input, 34.96 kWh stored energy, 14.72 kWh synthetic driving consumption and 80% final SOC. The thermal, price and electrochemical assumptions remain simplified by design.

7. Real SDK execution and independent numerical validation

Actually executed quantity	Verified outcome
Original persistent paceval graphs	23
True Multi-Graph-Batch result values	2,231,759
Actual Multi-Graph-Batch API calls	506
Scalar reusable-graph checks	23
Native C++ oracle result checks	116,380
Independent 80-digit mpmath oracle checks	2,760
Valid optimized charging schedules	36 / 36
Rejected deliberately invalid inputs	11 / 11
Original-SDK integration CTest	1 / 1 passed

Measured results, not performance or lifetime promises

Every execution figure above comes from the finished original-SDK run and its recorded result files. The independent high-precision checks re-evaluate the literal mathematical registry using mpmath with 80 decimal digits. The native C++ reference is a separate implementation of the specified formulas. They demonstrate computational consistency for the executed domain; they do **not** prove globally bounded errors or scientifically calibrated battery behavior.

There are intentionally no advertised processor speedups, native-vs-paceval accuracy-difference plots or claims of industrial cost savings in this public report. The numerical audit logs and complete formula inventory remain in the release package for reviewers.

8. Input tariffs and an optimized charging schedule

The first figure shows the six fixed, synthetic electricity price traces. The second uses the exact paceval-evaluated 24-hour path selected by the C++ optimizer for the “night discount” case (aging weight 8, objective version 1).

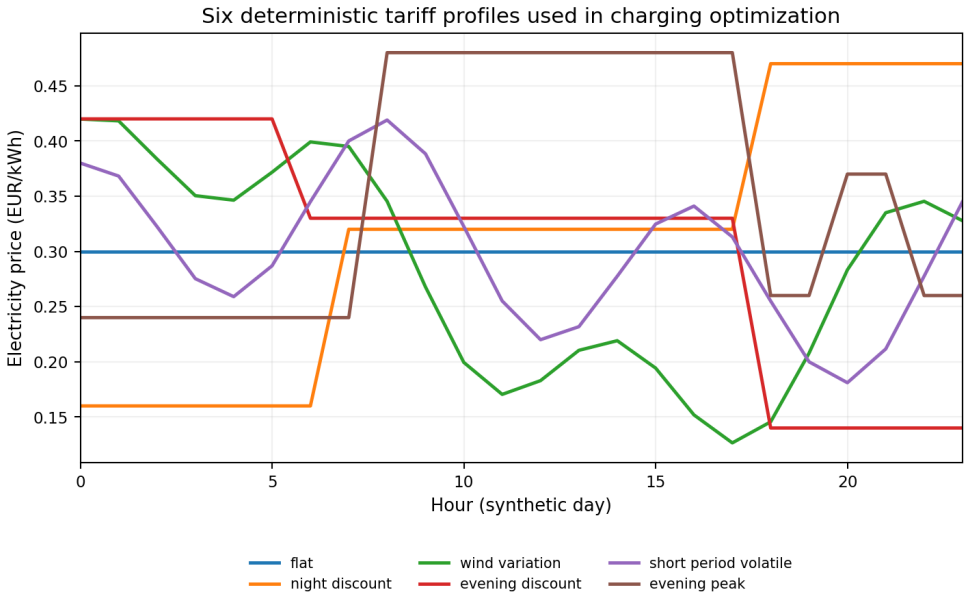


Figure 1. Six deterministic synthetic electricity tariffs. They are generated inputs, not provider prices or measured electricity market quotes.

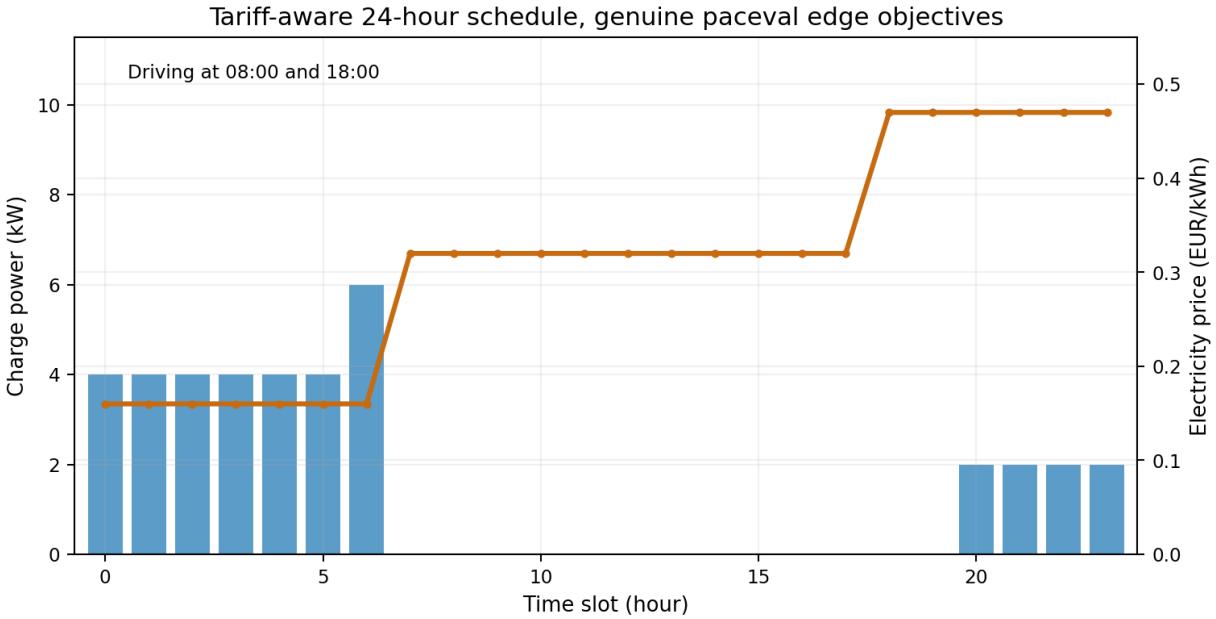


Figure 2. Optimized charging power and the corresponding input tariff. The mathematical edge values were computed by the original paceval SDK.

9. Composite charging cost: actual paceval surfaces

A 61-by-61 grid of charging power and starting SOC was evaluated in the actual SDK; only the visualization was created by matplotlib. The synthetic slot inputs are fixed at 0.31 EUR/kWh and 28°C, with a risk valuation of 8 EUR per synthetic risk unit.

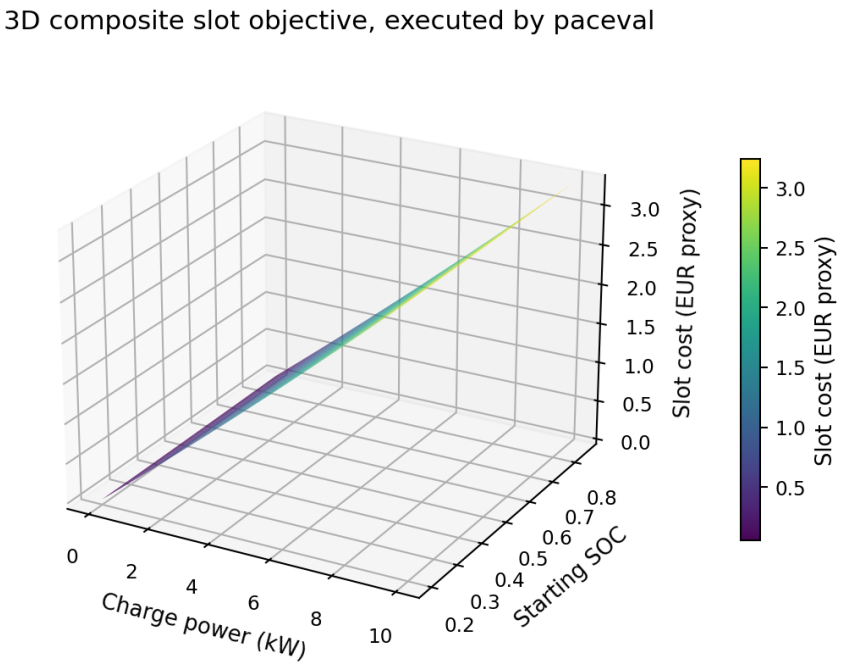


Figure 3. 3D composite cost surface, including tariff, surrogate aging valuation and power regularization.

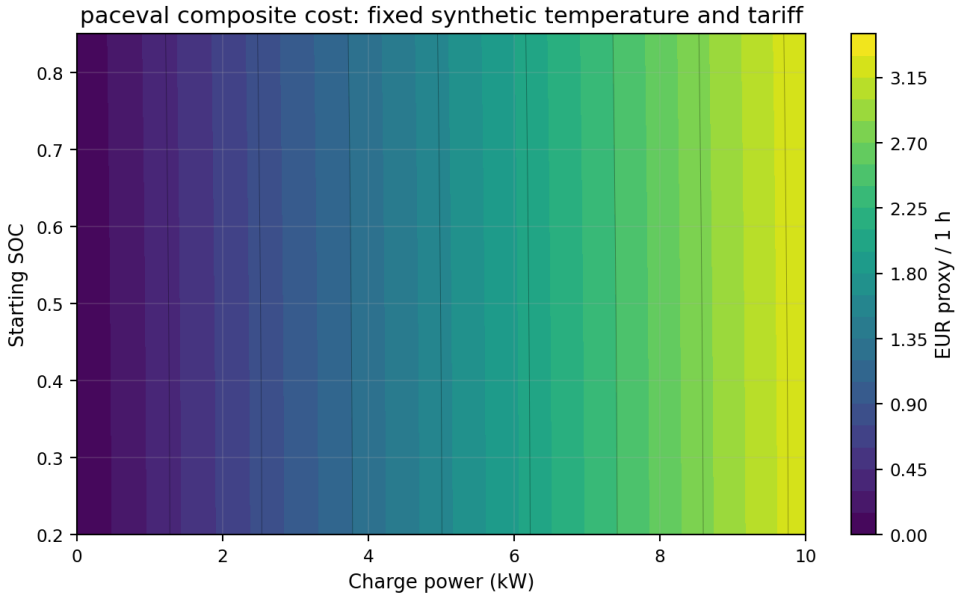


Figure 4. Contours of the identical paceval-evaluated mathematical objective; no interpolated measurements are claimed.

10. Synthetic degradation terms and sensitivity

The first response surface is a deliberately uncalibrated cycle/calendar risk proxy; it must not be interpreted as a measured capacity loss or expected lifetime. The second is a real graph-based analytic derivative of the mathematical version-1 objective.

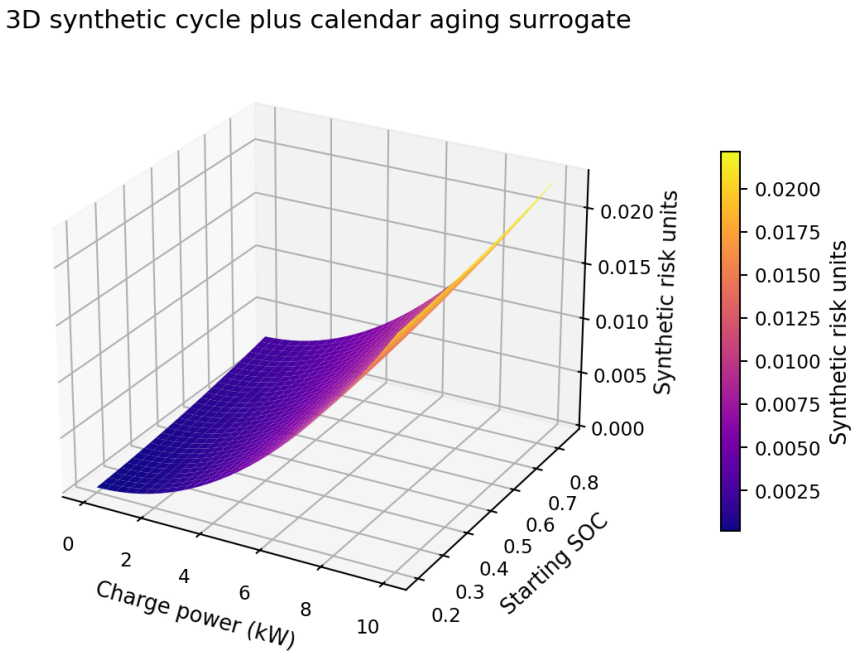


Figure 5. Synthetic aging-risk surface on a 61-by-61 grid, executed by paceval.

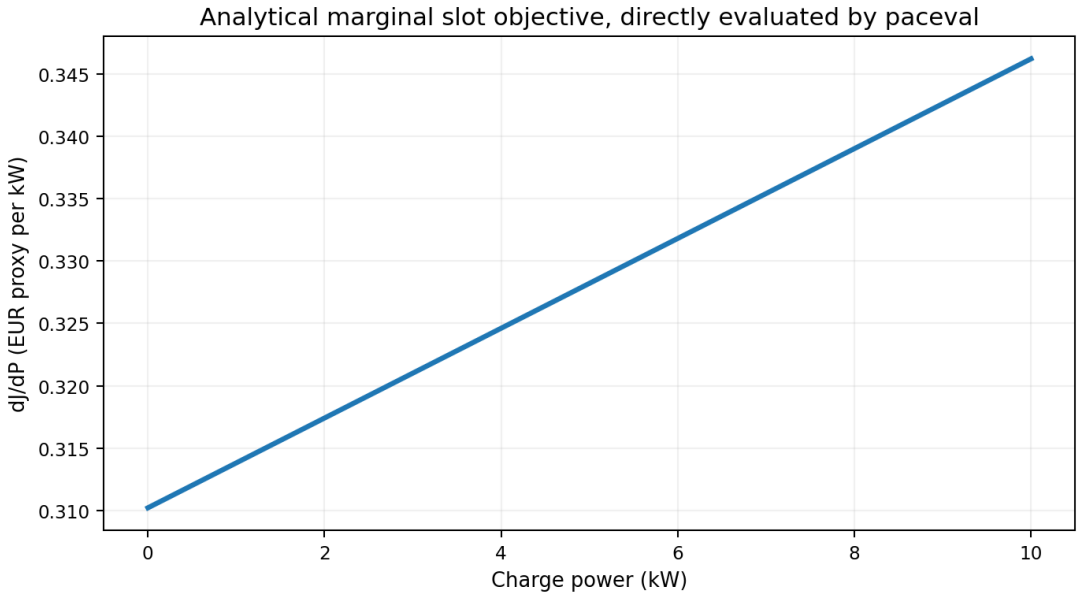


Figure 6. Exact analytic marginal objective with respect to charging power evaluated as a separate paceval computation handle.

11. Feasible SOC and charging decision outcomes

SOC trajectories are fully exported and audited against the selected paceval state transitions; the discrete scheduler enforces SOC safety, departure reserve and target equality. Charging is not permitted in the fixed driving interval.

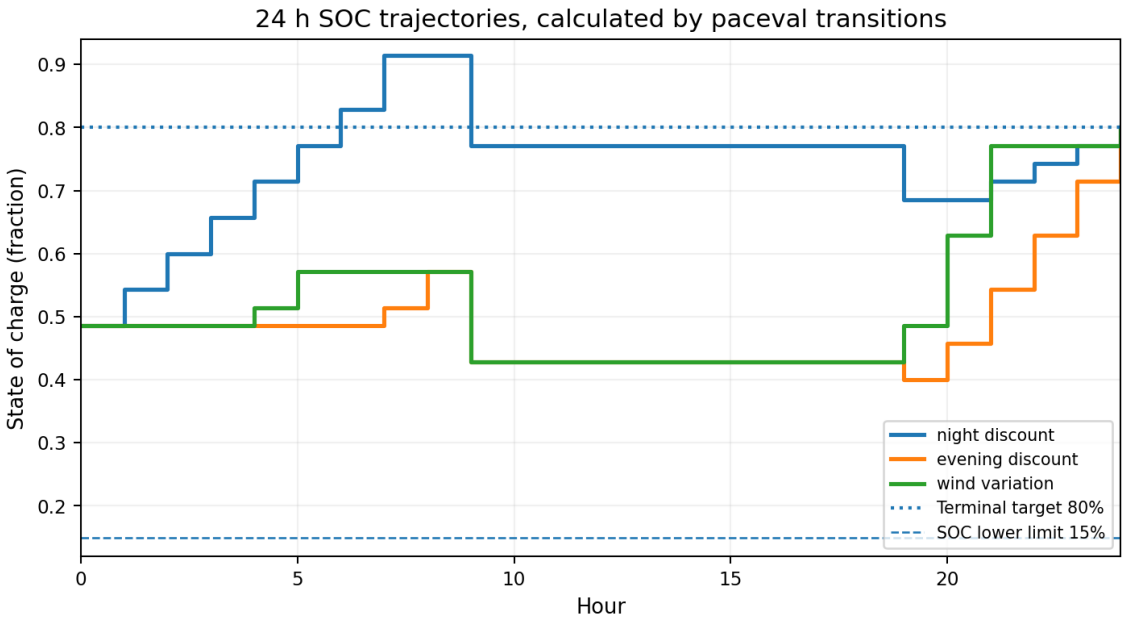


Figure 7. Selected SOC trajectories for three synthetic tariffs. All reach exactly 80% at the terminal hour.

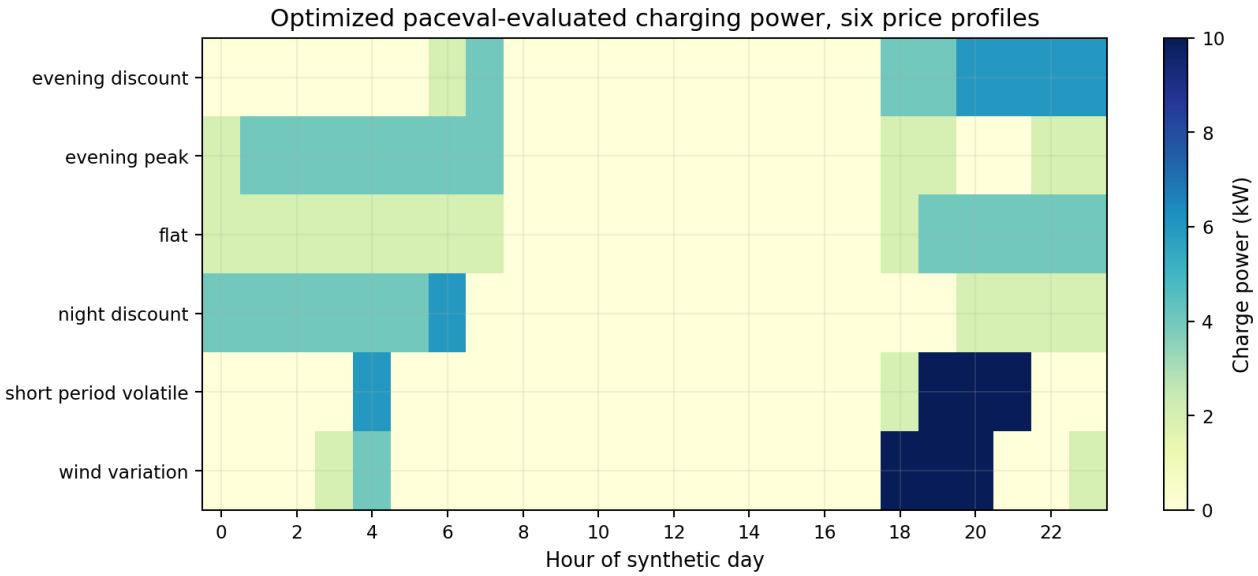


Figure 8. 24-hour charging-power decisions for all six price profiles (aging weight 8, version 1), after the actual mathematical batch evaluation.

12. Cost/risk tradeoffs and preserved formula versions

The horizontal axis represents synthetic electricity expense; the vertical axis is a dimensionless, uncalibrated stress index. The scatter is a mathematical tradeoff demonstration, not an estimate of battery lifetime or an achieved reduction in cell degradation.

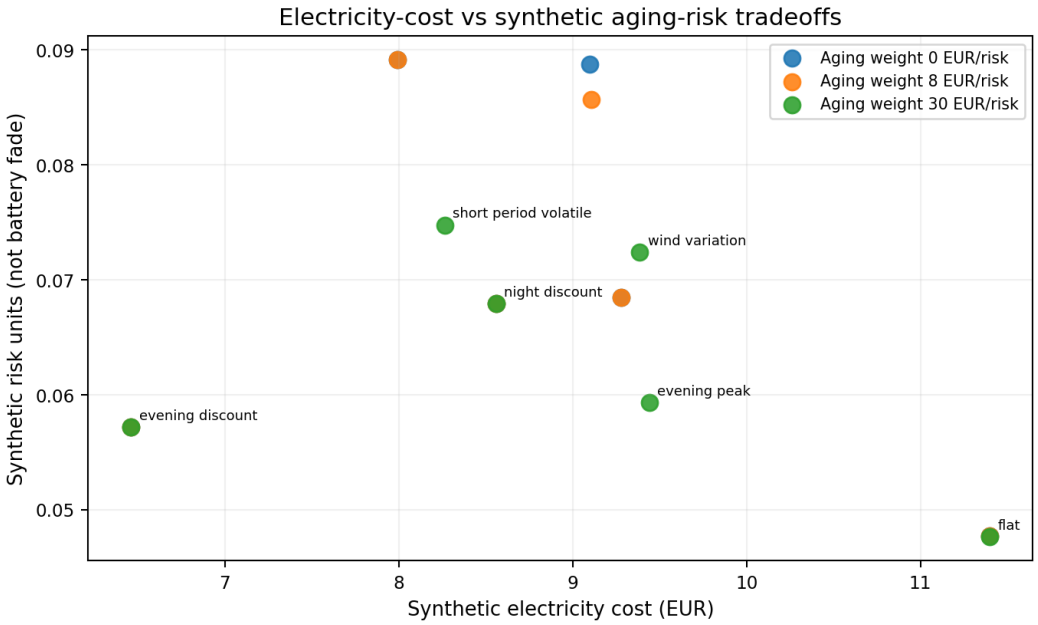


Figure 9. The chosen charging schedule changes as the assumed aging valuation changes, even when energy feasibility is held fixed.

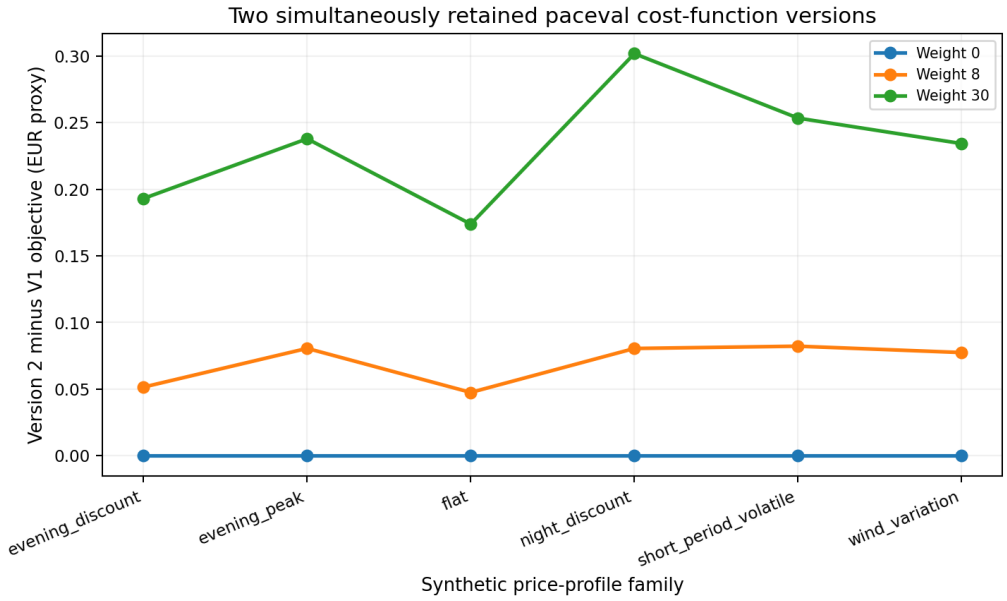


Figure 10. Version 2 adds an explicitly declared SOC factor and remains separately callable alongside version 1.

13. Scientific interpretation, reproducibility and next steps

Conclusion. This is a successful real paceval Mathematical Engine demonstration of reusable, versioned cost functions and constrained numerical state transitions inside an industrially motivated charging planner. The code produced real optimization results under an explicit synthetic energy/scheduling contract. It did not use an experimental electrochemical aging model, measured charging telemetry, live electricity price data or a commercial BMS.

Reproduce with original SDK files and independently provided runtime

From the GitHub package root, configure: **cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so**; build with **cmake --build build -j**; run **ctest --test-dir build --output-on-failure**; execute **./build/battery_charging_aging --out . --profiles 6 --weights 3 --surface 61**; then run the independent high-precision, scientific consistency, diagram and PDF scripts described in README.md.

Required next steps for a real industrial pilot

Integrate independently licensed and time-stamped electricity prices and actual driving schedules; validate the charger/BMS limits and vehicle telemetry; replace the nonphysical risk proxy with chemistry- and temperature-specific experimental aging data (for example, a BLAST-derived surrogate); and compare observed SOH trajectories under independent validation splits. This study does not pre-approve a charge-control strategy.

Selected scientific and technical sources

[1] Nnorom et al. (2026), Aging-Aware Battery Control via Convex Optimization. https://web.stanford.edu/~boyd/papers/aging_aware_battery_control.html

[2] NLR/NREL, Battery Lifespan and BLAST. https://www.nlr.gov/transportation/battery-lifespan

[3] NLR/NREL, BLAST battery modeling. https://nrel.sitefinity.cloud/transportation/blast

[4] Appleton & Fotouhi (2023), A Model-Based Battery Charging Optimization Framework. https://doi.org/10.1007/s42154-023-00221-8

[5] NLR/NREL EVI-Pro, charging demand and travel profiles. https://nrel.sitefinity.cloud/transportation/evi-pro

[6] paceval public C++ SDK / API. https://github.com/paceval/paceval

Source data: user-supplied WP_039 proposal (November 2026). All experiments use synthetic deterministic inputs. Output evidence: examples/ and results/ in this release.